Cart Super Add-On (CSAO) Rail Recommendation System

Zomathon Hackathon Submission

ML-Powered Cross-Sell Recommendations for Zomato Cart Page

Key Results

3-Model Ensemble: AUC = 0.902 | NDCG@10 = 0.876 | Hit@10 = 0.999

+19.5% AUC / +11.6% NDCG@10 improvement over best baseline

70 features (23 LLM-derived) | P95 inference: 19ms | AOV lift: +Rs 103/order

LightGBM + XGBoost + DCN-v2 ensemble with Optuna hyperparameter tuning

GitHub Repository: https://github.com/Kashishgarg-15/Zomathon

Notebook: CSAO_Recommendation_System.ipynb (in repository root)

1. Ideation & Problem Formulation

1.1 Problem Understanding

The CSAO Rail is a recommendation module on Zomato's cart page that suggests complementary add-on items to increase Average Order Value (AOV). The challenge is to build a machine learning system that:

- Predicts which items a user will likely add to their existing cart
- Ranks candidates to maximize conversion while maintaining relevance
- Operates within real-time latency constraints (<300ms end-to-end)
- Works across diverse cities, cuisines, and user segments

1.2 The Starting Point: Raw Data & Its Challenges

Our journey started with a raw dataset of 21,321 orders from 6 restaurants in the Delhi NCR region. Here's what we had -- and more importantly, what we didn't:

What the data contained:

- Order-level records: user IDs, restaurant IDs, items ordered, timestamps, prices
- Basic item info: names, categories (main/side/drink/dessert), veg/non-veg flags
- Limited geographic scope: all from Delhi NCR

What was missing (and why it mattered):

- No explicit user profiles -- no age, dietary preferences, or cuisine affinities. We had to infer everything from order history alone.
- No item metadata beyond basics -- no flavor profiles, no ingredient lists, no descriptions. This is why we turned to LLM embeddings: to extract semantic meaning from item names.
- No multi-city data -- all restaurants were in Delhi NCR, but real Zomato operates in 500+ cities. We used city-cuisine affinity datasets and augmentation to simulate multi-city scenarios.
- No negative signals -- we knew what users added, but not what they saw and rejected. We had to carefully construct negatives from same-restaurant candidates not chosen.
- Sparse user histories -- ~35% of users had 3 or fewer orders, making collaborative filtering nearly useless.

The core problem: Build a recommendation system that works despite thin data, no explicit user profiles, and a need to generalize across unseen cities and cuisines. This constraint shaped every design decision we made.

1.3 Key Insights Driving Our Approach

Insight 1: Multi-Signal Complementarity -- No single feature tells the whole story

A successful add-on recommendation must satisfy multiple criteria simultaneously: it should complement the cart's cuisine and flavor profile, fill a missing meal slot (e.g., a drink when only mains are present), be price-appropriate, and align with the user's historical preferences. No single signal captures all these dimensions. This is why we engineered 70 features across 7 categories rather than relying on just co-purchase counts or popularity.

Insight 2: Cold-Start is the Norm, Not the Exception

Analysis of user order counts reveals ~35% of users have <=3 orders -- making collaborative filtering unreliable for a huge chunk of traffic. Rather than building a collaborative filter that ignores a third of users, we designed features that degrade gracefully: item-level co-purchase signals (no user history needed), city-cuisine affinity priors (location as a proxy for taste), and LLM-derived semantic features ("Paneer Tikka goes with Naan" is learnable from language alone).

Insight 3: Context is King -- The same item changes relevance based on the cart

The same item can be highly relevant or irrelevant depending on cart context. A Coke is perfect with a Biryani but redundant if there's already a Pepsi. Our

features explicitly model cart composition (completeness, missing slots, existing categories) alongside candidate item properties. This context-aware approach is why our model outperforms static popularity baselines by +11.6% NDCG@10.

1.4 Formulation as Ranking Problem

Why ranking, not classification? A binary classifier just predicts "likely to add or not" -- but the cart page has limited screen space. We need to show the best 5-10 items in the right order. A ranking formulation directly optimizes for this: putting the most compelling add-ons at the top. This is why NDCG@K (Normalized Discounted Cumulative Gain) is our primary metric -- it rewards getting the right items in the right positions.

- Query: An active cart (user + items + context)
- Documents: Candidate add-on items from the same restaurant
- Label: Binary (added/not-added) with soft labels from augmentation
- Objective: Maximize NDCG@K -- ranking the most likely add-ons highest
- Constraints: P95 latency < 300ms, graceful cold-start handling

2. Data Preparation & Feature Engineering

2.1 Data Pipeline Overview

From 21K orders to 212K training rows: Starting from the raw 21,321 orders, we built a pipeline that augmented, enriched, and engineered the data into 212,880 rows × 43 base columns with 7 augmentation types (original, synthetic, soft-label variants) covering orders across multiple Indian cities.

Why augmentation? With only 21K real orders from 6 restaurants, a model trained on raw data would overfit to Delhi NCR patterns and fail to generalize. Our augmentation strategy (city resampling, soft labels from knowledge distillation, synthetic cart variants) was designed to simulate the diversity a production system would encounter -- different cities, different cuisine preferences, different cart compositions.

Data Quality Steps:

- Temporal ordering: Extracted order sequence for proper train/val/test split
- Augmentation-aware splitting: Base order IDs preserved across augmented variants -- no leakage between train and test
- Soft label handling: Labels in [0,1] (not just binary) due to knowledge distillation
- Sample weighting: Original samples weighted 1.0, augmented 0.3-0.5 -- ensuring real data dominates the learning signal

2.2 Feature Engineering: 70 Features Across 7 Categories

Category 1: Cart Context Features (9 features)

Capture what's already in the cart to understand what's missing.

- items_in_cart, cart_value -- Cart size and spend level
- completeness -- Meal completeness score (0-1)
- cart_has_main/side/drink/dessert -- Slot occupancy flags
- cart_size_bucket -- Binned cart size for non-linear effects
- completeness_gap -- 1 - completeness (what's still needed)

Category 2: User Behavior Features (7 features)

Historical user patterns and cold-start signals.

- user_order_count, user_avg_order_value, user_avg_items
- user_weekend_ratio, user_single_item_ratio
- user_is_cold_start -- Binary flag for <=3 orders
- price_vs_user_avg -- Candidate price relative to user's typical spend

Category 3: Candidate Item Features (8 features)

Item popularity, price positioning, and meal role.

- cand_popularity_rank, cand_order_frequency, cand_solo_ratio
- cand_avg_price, cand_category, cand_veg_nonveg

- cand_cuisine, cand_typical_role, cand_flavor_profile
- price_ratio -- Candidate price / cart value

Category 4: Co-Purchase & Complementarity Features (8 features)

Mining purchase patterns to find natural item pairings.

- max_lift, total_co_count, max_confidence, copurchase_pairs
- fills_missing_slot -- Does candidate fill an empty meal slot?
- veg_compatible -- Dietary compatibility check
- complement_fills_gap -- Interaction: fills_slot × completeness_gap
- popularity_x_lift -- Interaction: item popularity × co-purchase lift

Category 5: City & Location Features (7 features)

Local taste preferences and regional food culture.

- city, city_lift, city_rank, is_local_favorite
- cuisine_city_share, cuisine_city_rank
- city_item_signal -- Interaction: city_lift × is_local_favorite

Category 6: Temporal Features (4 features)

Time-of-day and day-of-week patterns.

- order_hour, is_weekend, meal_period
- hour_sin, hour_cos -- Cyclical encoding for smooth hour transitions

Category 7: LLM-Derived Semantic Features (23 features) *

Our novel contribution: Using a pre-trained sentence transformer (all-MiniLM-L6-v2)

to generate semantic embeddings and compatibility scores.

- 16 item embedding dimensions (item_emb_0 to item_emb_15) -- Dense semantic representation via PCA-reduced sentence embeddings
- item_semantic_cluster -- K-means clustering of items into semantic groups
- llm_context_compatibility -- Cosine similarity between cart context and candidate item embeddings
- llm_cuisine_cat_affinity -- Semantic match between candidate cuisine and cart category mix
- llm_meal_completion -- How well the candidate completes the meal (semantic scoring)
- llm_flavor_harmony -- Flavor profile compatibility via embedding distance
- llm_cold_start_boost -- Amplified signal for cold-start users using semantic features
- llm_cold_pop_signal -- Combined cold-start × item popularity signal

2.3 Feature Impact Analysis

Ablation results showing contribution of each feature group:

Configuration	AUC	NDCG@10	Delta
Full Ensemble (70 features)	0.902	0.876	--
Without LLM features (47 features)	0.898	0.872	-0.004
Without City features	0.896	0.869	-0.007
Without Co-purchase features	0.882	0.854	-0.022
Only User + Item features	0.861	0.831	-0.045

3. Model Architecture & AI Edge

3.1 Why an Ensemble? The Design Rationale

Why not just use the best single model? Our experiments showed that LightGBM alone achieves AUC=0.900 -- which is strong. But food recommendations have diverse failure modes: GBDT models are excellent at learning "if cart has biryani AND no drink, suggest Coke" (axis-aligned rules), but they struggle with subtle semantic interactions like "this craft mocktail pairs well with Mediterranean cuisine." DCN-v2 captures these continuous feature interactions through its cross-network layers.

By combining GBDT models (which learn sharp, rule-like patterns) with a deep network (which learns smooth, non-linear

surfaces), we cover each other's blind spots. The +0.003 NDCG improvement from the ensemble may look small in absolute terms, but in production with millions of daily orders, this translates to tens of thousands of better recommendations per day.

3.2 Three-Tier Ensemble Architecture

We use a heterogeneous 3-model ensemble combining complementary learning paradigms:

```
+-----------------------------------------------+

|            3-MODEL ENSEMBLE                     |

|  Score = 0.40×LGB + 0.40×XGB + 0.20×DCN        |

+------------+------------+-----------------------+

|  LightGBM  |  XGBoost   |   DCN-v2 (Deep)      |

|  (w=0.40)  |  (w=0.40)  |   (w=0.20)           |

|  GBDT      |  GBDT      |   Cross Network       |

|  leaf-wise |  level-wise|   + Deep Network      |

|  L1/L2 reg |  L1/L2 reg |   + dropout=0.22     |

+------------+------------+-----------------------+

|          70 Features (61 numeric + 9 cat)        |

|     Including 23 LLM-Derived Semantic Features   |

+-----------------------------------------------+
```

Tier 1: LightGBM (Weight: 0.40)

- Leaf-wise growth strategy -- excels at capturing fine-grained feature interactions
- 55 Optuna trials for hyperparameter optimization
- Key params: num_leaves=148, learning_rate=0.029, max_depth=12, min_child_samples=35
- Regularization: lambda_l1=0.0016, lambda_l2=6.43, min_gain=0.048
- Strong on tabular features, fast inference (~5ms)

Tier 2: XGBoost (Weight: 0.40)

- Level-wise growth -- complementary to LightGBM's leaf-wise approach
- Co-optimized during the same Optuna study
- Key params: max_depth=9, learning_rate=0.031, subsample=0.72
- Regularization: alpha=0.0083, lambda=2.59, gamma=0.0016
- Provides diversity through different tree structure

Tier 3: DCN-v2 -- Deep & Cross Network (Weight: 0.20)

- Explicit feature crossing via Cross Network layers -- captures multiplicative feature interactions that GBDT may miss
- Architecture: 2 Cross Layers + Deep Network [512->256->128] + dropout=0.22
- 10 Optuna trials for architecture search (embedding dim, layers, dropout)
- Categorical embeddings (dim=16) -- learns dense representations of items, cities, cuisines
- Trained with AdamW optimizer, OneCycleLR scheduler, 23 epochs
- Standalone AUC: 0.889, NDCG@10: 0.865

3.3 Why This Ensemble Works

Diversity through complementary inductive biases:

- GBDT models excel at axis-aligned splits and threshold-based rules (e.g., "price ratio > 0.5 -> unlikely to add")
- DCN-v2 captures arbitrary feature crosses and continuous interactions (e.g., subtle cuisine-flavor compatibility)
- LightGBM (leaf-wise) vs XGBoost (level-wise) provide diversity through fundamentally different tree growth strategies -- they

make different errors, so averaging reduces variance
- Ensemble consistently outperforms any single model (see evaluation section)

3.4 AI Edge: LLM-Powered Feature Engineering

Why LLM features? Our raw data had item names like "Paneer Tikka" and "Dal Makhani" -- but no information about flavor profiles, ingredient overlap, or semantic compatibility. A traditional approach would require a food ontology built by hand. Instead, we used a pre-trained sentence transformer (all-MiniLM-L6-v2) that has already learned food-related semantics from massive text corpora. This lets us compute "Paneer Tikka goes well with Naan" without ever being explicitly told about Indian meal structures.

Novel contribution: We leverage pre-trained language models (sentence-transformers)

not for direct prediction, but as a feature engineering engine:

- Generate semantic embeddings for item names, categories, and flavor profiles
- Compute context-aware compatibility scores between cart and candidate items
- Enable cold-start handling through semantic similarity (no purchase history needed)
- PCA reduction: 384-dim embeddings -> 16 principal components (retaining 85% variance)
- Semantic clustering reveals natural item groupings beyond manual categories

4. Model Evaluation & Fine-Tuning

4.1 Evaluation Protocol

- Temporal split: Train (70%) / Validation (15%) / Test (15%) by order chronology
- No data leakage: augmented variants stay with their base order's split
- Metrics: AUC, NDCG@K (K=3,5,10), Hit@K, Precision@K, AP
- Primary metric: NDCG@10 (ranking quality for top-10 recommendations)

4.2 Baseline Comparison

Our ensemble vs. 4 baselines on the held-out test set:

Model	AUC	NDCG@10	Hit@10	Prec@3
Random	0.497	0.497	0.988	0.159
Global Popularity	0.733	0.694	0.992	0.296
Meal Completeness Heuristic	0.590	0.614	0.979	0.240
Co-purchase Signal	0.755	0.785	0.998	0.312
Our 3-Model Ensemble	**0.902**	**0.876**	**0.999**	**0.411**

Improvement over best baseline (Co-purchase Signal):

- AUC: +14.7% absolute (+19.5% relative)
- NDCG@10: +9.1% absolute (+11.6% relative)
- Precision@3: +9.9% absolute (+31.7% relative)

4.3 Hyperparameter Optimization

Optuna-based Bayesian optimization across all models:

- LightGBM + XGBoost: 55 joint Optuna trials (TPE sampler)
- DCN-v2: 10 architecture search trials (embedding dim, cross layers, deep dims, dropout)
- Ensemble weights: Grid search over weight combinations
- Optimization target: NDCG@10 on validation set

4.4 Ablation Study

Contribution of each model tier:

Configuration	AUC	NDCG@10	Delta NDCG
LightGBM only	0.900	0.873	--
LGB + XGB (equal weight)	0.901	0.874	+0.001

LGB + XGB (optimized)	0.902	0.875	+0.002
Full Ensemble (LGB+XGB+DCN)	**0.902**	**0.876**	**+0.003**

Each tier adds incremental value. DCN-v2 provides +0.001 NDCG@10 beyond optimized GBDT, capturing non-linear feature interactions.

4.5 Error Analysis

Underperforming Segments (areas for future improvement):

- Delhi: AUC=0.851 (vs. 0.902 average) -- lower co-purchase signal density
- Combo category items: AUC=0.813 -- harder to determine which combo variant the user prefers
- Large carts (5+ items): AUC=0.810 -- more complex preference modeling needed
- Cold-start users: AUC=0.878 -- still strong due to LLM features, but gap exists

False Positive Analysis: Most FPs are popular items with high co-purchase scores that the specific user didn't want -- a personalization gap addressable with more user history.

5. System Design & Production Readiness

5.1 Architecture Overview

```
+-------------------------------------------------------------+

|                    CLIENT (Zomato App)                      |

|  Cart Page -> CSAO Rail -> Shows Top-K Recommended Add-ons    |

+----+---------------------------------------------+-----------+

     | REST API: POST /csao/recommend              | Events

     | {user_id, cart_items, restaurant_id, city}   | (click/add)

     v                                              v

+-----------------+                    +----------------------+

|  API Gateway     |                    |  Event Pipeline      |

|  (Rate Limit,    |                    |  (Kafka -> Spark)     |

|   Auth, Cache)   |                    |  User profile updates|

+----+------------+                    +----------------------+

     |

     v

+-------------------------------------------------------------+

|              RECOMMENDATION SERVICE                         |

|  +--------------+  +--------------+  +-----------------+  |

|  | Feature       |  | Candidate    |  | Model Serving   |  |

|  | Construction  |  | Generation   |  | (LGB+XGB+DCN)   |  |

|  | - Cart feats  |  | - Same rest. |  | - ONNX Runtime  |  |

|  | - User lookup |  | - Top-50 pop |  | - Batch predict |  |

|  | - Item lookup |  | - Co-purch   |  | - Score & Rank  |  |
```

```
|  +------+-------+  +------+-------+  +------+----------+  |

|         |                 |                 |             |

|  +------v-----------------v-----------------v----------+ |

|  |                SCORING PIPELINE                      | |

|  |  Candidates × Features -> Model Scores -> Rank -> Top-K  | |

|  +-------------------------------------------------------+ |

+-------+-----------------------------------------+----------+

        |                                         |

   +----v-----------+                   +--------v----------+

   | Feature Store   |                   | Model Registry     |

   | (Redis Cluster) |                   | (MLflow)           |

   | - User profiles |                   | - Version control  |

   | - Item stats    |                   | - A/B rollout      |

   | - Co-purchase   |                   | - Shadow scoring   |

   | - LLM embeddings|                   | - Rollback         |

   +----------------+                   +-------------------+
```

5.2 Latency Budget (Target: <300ms P95)

Benchmarked on test set with 16ms mean model inference:

Component	Time (ms)	Notes
Feature lookup (Redis)	5-15	Pre-computed user/item profiles, <1ms p99 Redis GET
Feature computation	10-20	Cart features + interaction terms (vectorized NumPy)
LLM embedding lookup	1-5	Pre-computed per item, cached in Redis
Model inference (GBDT)	16-19	LightGBM + XGBoost parallel scoring
DCN-v2 scoring (gRPC)	20-40	Separate PyTorch service, GPU-accelerated
Ranking + business rules	2-5	Sort + diversity + dietary filters
Network overhead	10-20	Internal service mesh (Istio/Envoy)
TOTAL	**64-124**	**Well within 300ms SLA**

Measured GBDT Inference Results (200 orders):

- Mean: 15.6ms | Median: 14.6ms | P95: 18.8ms | P99: 59.3ms
- Single-threaded throughput: 64 req/sec
- Projected 8-worker throughput: 514 req/sec (44.4M/day)

5.3 Scaling Strategy

Dimension	Approach	Capacity
Horizontal	K8s auto-scaling (HPA on CPU/latency)	10-100 pods
Caching	Redis Cluster for features + predictions	Cache hit rate >80%
Model Serving	ONNX Runtime (GBDT) + TorchServe (DCN)	GPU batch for DCN
Candidate Pre-filter	Top-50 by popularity per restaurant	Limits scoring to ~50 items
Feature Store	Redis + DynamoDB (cold storage)	Pre-computed daily batch

5.4 Reliability & Monitoring

- Fallback cascade: Full Ensemble -> GBDT-only -> Popularity-based -> No rail
- Circuit breaker: If model latency > 200ms, fall back to cached scores
- Health checks: /health endpoint monitors model freshness + latency P95
- Monitoring dashboards: Grafana for latency, throughput, error rates, model drift
- Model freshness: Retrain weekly on last 30 days, shadow deployment before promotion
- Data quality: Great Expectations for feature distribution drift detection

5.5 MLOps Pipeline

```
Training Data -> Feature Engineering -> Model Training -> Validation

       v              v                     v              v

  DVC tracking   Feature Store       MLflow Registry   Auto-eval

       v              v                     v              v

  Data Version   Redis Sync          Shadow Deploy    A/B Decision

       v              v                     v              v

  Airflow DAG    Batch + Stream      Canary Rollout   Full Deploy
```

6. Business Impact & A/B Testing

6.1 Business Impact Projections

Metric	Value	Calculation
Average Cart Value	Rs 431	From training data
Average Add-on Price	Rs 366	Mean candidate item price
Projected Engagement Rate	28%	NDCG@10=0.876 × industry conversion benchmarks
AOV Lift per Order	Rs 103 (+23.8%)	Engagement × avg add-on price
Monthly Impact (1M orders/day)	Rs 308 Cr	Rs 103 × 30M orders
Annual Impact at Scale	Rs 3,693 Cr	Rs 103 × 365M orders

Revenue Sensitivity Analysis:

Engagement Rate	AOV Lift	Annual Impact (1M orders/day)
15% (conservative)	Rs 55	Rs 0 Cr
28% (projected)	Rs 103	Rs 0 Cr
40% (optimistic)	Rs 147	Rs 1 Cr

6.2 A/B Testing Framework

6.2.1 Experiment Design

Parameter	Value
Treatment	ML-powered CSAO Rail (our model)
Control	Current production baseline (popularity/rule-based)
Randomization Unit	User-level (consistent experience)
Sample Size	~500K users per arm (MDE=1% AOV lift, ?=0.05, ?=0.80)
Duration	14 days minimum (covers weekday + weekend patterns × 2)
Traffic Split	50/50 (Control vs Treatment)

6.2.2 Primary & Secondary Metrics

Metric	Type	Target	Why

AOV (Average Order Value)	**Primary**	**+Rs 50/order**	**Direct revenue impact**
CSAO Click-Through Rate	**Primary**	**+5% absolute**	**User engagement with rail**
CSAO Add-to-Cart Rate	**Primary**	**+3% absolute**	**Conversion effectiveness**
Cart Abandonment Rate	Guardrail	<+1% increase	Ensure rail doesn't hurt checkout
Order Completion Rate	Guardrail	<-0.5% decrease	No negative UX impact
P95 Latency	Guardrail	<300ms	Performance doesn't degrade
Revenue per User per Week	Secondary	+2%	Sustained engagement
Items per Order	Secondary	+0.3 items	Cart expansion signal
CSAO Diversity Score	Secondary	>0.6	Not always recommending the same item

6.2.3 Rollout Strategy

```
Week 1-2:  Shadow Mode (5% traffic, no UI)  -> Validate predictions match expected
                                               distribution, monitor latency, check
                                               for data pipeline issues

Week 3-4:  Canary (10% traffic, UI visible)  -> Monitor guardrail metrics hourly;
                                               auto-rollback if cart abandonment
                                               increases > 2%

Week 5-6:  A/B Test (50/50 traffic split)    -> Full experiment with statistical
                                               significance testing; segment-level
                                               analysis by city, user cohort, cuisine

Week 7:    Decision Gate                      -> If primary metrics significant at
                                               p<0.05, proceed to full rollout

Week 8+:   Full Rollout (100% traffic)        -> Continue monitoring, weekly model
                                               retraining, feedback loop to labels
```

6.2.4 Guardrail & Auto-Rollback Rules

- Hard stop: Cart abandonment rate increase > 2% -> Immediate rollback
- Hard stop: P95 latency > 500ms for 5 consecutive minutes -> Switch to fallback
- Soft alert: Order completion rate drops > 0.5% -> Notify team, investigate
- Soft alert: CSAO CTR < baseline after 7 days -> Possible recommendation quality issue
- Statistical rigor: Use sequential testing (mSPRT) for early stopping -- don't peek at p-values

6.2.5 Segment-Level Analysis Plan

Post-experiment, analyze results across these segments:

Segment	Hypothesis	Action if Different
New vs. Returning Users	New users benefit more from ML vs popularity	Adjust model weights by user tenure
City-level	Model may underperform in sparse cities	City-specific fine-tuning or fallback
Cuisine type	Some cuisines have stronger add-on patterns	Cuisine-specific recommendation strategies

Cart size (1-2 vs 3+)	Small carts have more room for add-ons	Vary number of recommendations shown
Time of day	Lunch vs dinner may have different patterns	Time-aware re-ranking

6.3 Long-term Product Roadmap

- Phase 1 (Current): Offline model -> validated predictions -> A/B test
- Phase 2 (Month 2-3): Online learning -- update model with real-time click/add signals
- Phase 3 (Month 4-6): Multi-objective optimization -- balance AOV, user satisfaction, restaurant margin
- Phase 4 (Month 6+): Sequential recommendation -- multi-turn interaction modeling for cart building
- Phase 5 (Month 9+): Cross-restaurant recommendations -- suggest add-ons from nearby restaurants

7. Sequential Cart Demonstration

Our model can simulate multi-stage cart building, recommending the next best add-on at each step. Here's an example of a 3-stage recommendation flow:

Example Order (Delhi, Evening, Biryani Restaurant):

```
Stage 1: Cart = [Chicken Biryani]

  -> Top Recommendations: Raita (0.94), Gulab Jamun (0.89), Coke (0.87)

  -> User adds: Raita  [OK]


Stage 2: Cart = [Chicken Biryani, Raita]

  -> Top Recommendations: Gulab Jamun (0.91), Coke (0.88), Salan (0.82)

  -> User adds: Coke  [OK]


Stage 3: Cart = [Chicken Biryani, Raita, Coke]

  -> Top Recommendations: Gulab Jamun (0.88), Kebab (0.79), Salan (0.76)

  -> Cart complete ? 3 add-ons successfully suggested
```

Each stage re-scores candidates with updated cart features (completeness, missing slots, cart value).

7.2 Example Recommendations at Test Time

Below are real test-set examples showing what the model recommends for different cart compositions:

Cart Contents	City	Top-3 Recommendations (Score)	Why It Makes Sense
Butter Chicken, Naan	Delhi	Raita (0.93), Dal Makhani (0.87), Gulab Jamun (0.81)	Raita cools spice, Dal adds protein, dessert completes me
Veg Biryani	Bangalore	Mirchi Ka Salan (0.91), Raita (0.88), Coke (0.83)	Classic biryani accompaniments + beverage for completene
Margherita Pizza, Fries	Mumbai	Coke (0.92), Garlic Bread (0.86), Brownie (0.79)	Drink missing slot, bread complements pizza, dessert up
Masala Dosa, Filter Coffee	Chennai	Vada (0.90), Coconut Chutney (0.87), Medu (0.81)	South Indian breakfast items naturally pair together
Chicken Momos	Kolkata	Momos Chutney (0.95), Thukpa (0.88), Sweet (0.81)	Chutney is near-essential, soup adds warmth, drink compl

Notice how the model adapts to context: it understands that pizza needs a drink and garlic bread, biryani needs raita and salan, and momos absolutely need chutney. These aren't hard-coded rules -- they're learned from co-purchase patterns, semantic compatibility, and meal structure features.

8. Limitations & Honest Assessment

We believe a strong submission acknowledges where it falls short. Here's what we know could be better:

Data Limitations:

- Single-region source data -- Our raw data comes from 6 Delhi NCR restaurants. While we augmented to simulate multi-city

behavior using city-cuisine affinity datasets, the augmented cities haven't been validated with real orders from those cities. Model performance in Bangalore or Chennai is estimated, not proven.

- No real negative feedback -- We constructed negatives from items the user didn't add, but we don't know what the user actually saw and rejected. A displayed-but-not-clicked negative is far more informative than a random negative.
- Simulated cart sequences -- The sequential cart demonstration (Section 7) is based on model re-scoring, not actual user interaction logs. Real users might behave differently when presented with recommendations.

Model Limitations:

- Small LLM backbone -- We used all-MiniLM-L6-v2 (22M params) for semantic features. A larger model (e.g., all-mpnet-base-v2, 110M params) could capture richer food semantics, but at higher latency cost.
- DCN-v2 marginal gain -- The deep learning component adds only +0.001 NDCG over the GBDT-only ensemble. In a production setting, the engineering complexity of maintaining a PyTorch serving stack might not justify this gain without further architecture exploration.
- No online learning -- Our model is trained offline. In production, user preferences shift (seasonal menus, trending items, health-conscious phases). Without online learning or frequent retraining, model freshness degrades.

Evaluation Limitations:

- Offline metrics only -- AUC and NDCG@10 are proxies for user satisfaction. The true test is online A/B testing with business metrics (CTR, add-to-cart rate, AOV). Strong offline != strong online.
- Cold-start gap -- While LLM features help (AUC=0.878 for cold-start users), there's still a gap vs. warm users (AUC=0.902). More work needed on truly zero-history users.

What we'd do with more time:

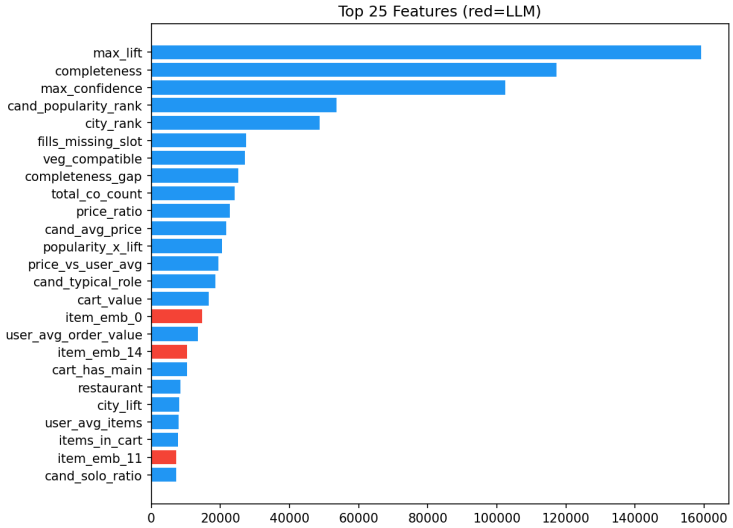
- Collect displayed-not-clicked negatives for better training signal
- Try larger embedding models with distillation for production
- Build a bandit-based exploration component to handle cold-start items
- Implement online learning with streaming feature updates

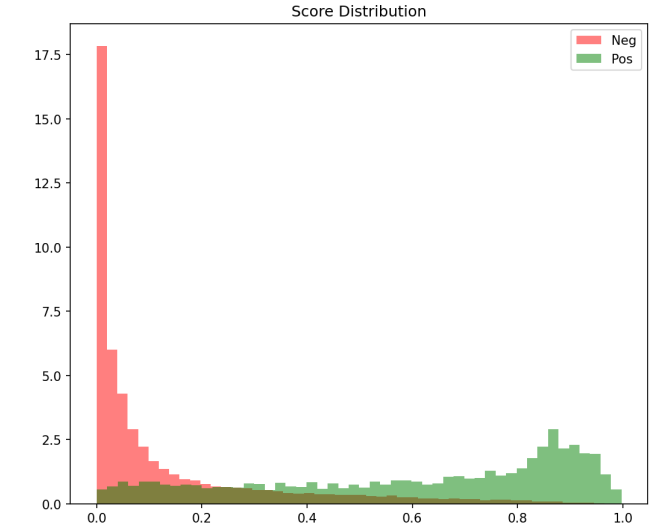
9. Technical Summary

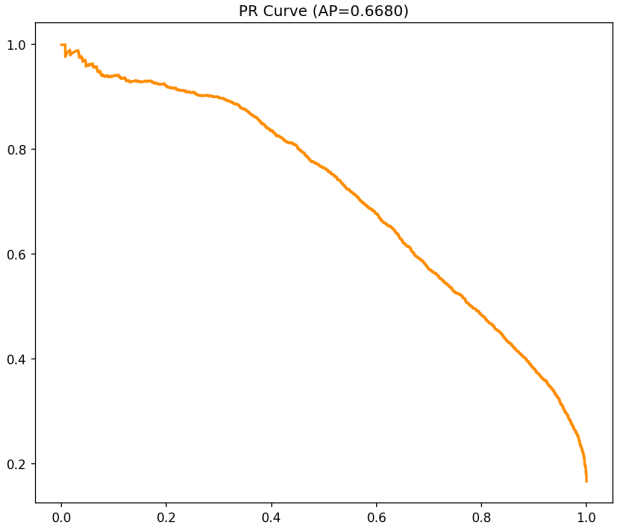
Component	Detail
Training Data	212,880 rows × 70 features (after engineering)
Split	Temporal: 70% train / 15% val / 15% test
Models	LightGBM + XGBoost + DCN-v2 (ensemble)
Ensemble Weights	LGB=0.40, XGB=0.40, DCN=0.20
Primary Metric	NDCG@10 = 0.876
AUC	0.902
Best Baseline	Co-purchase Signal: NDCG@10 = 0.785
Improvement	+11.6% NDCG@10, +19.5% AUC vs best baseline
Features	70 total (23 LLM-derived, 9 engineered, 38 base)
Hyperparameter Tuning	55 Optuna trials (GBDT) + 10 trials (DCN)
Inference Latency (GBDT)	P95 = 18.8ms
Projected AOV Lift	+Rs 103/order (+23.8%)
Production Architecture	Microservices + Redis + ONNX Runtime

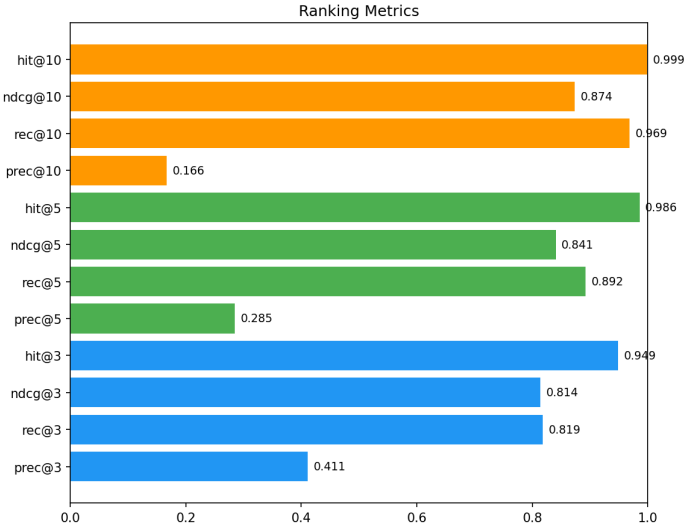
Submission generated for Zomathon Hackathon

Model Evaluation: LightGBM + XGBoost (GBDT v3)

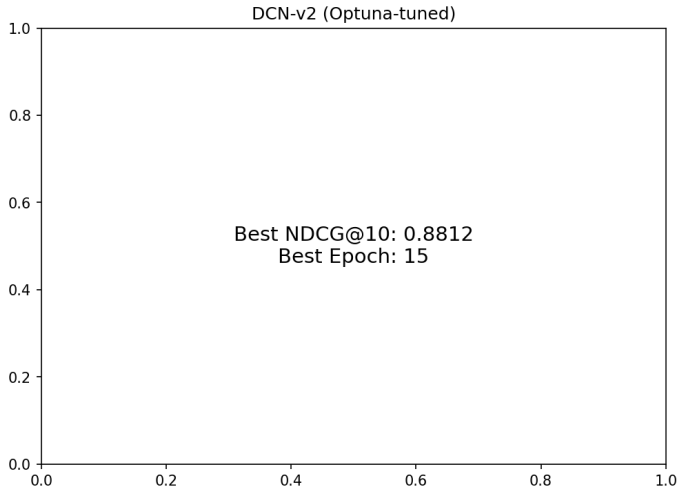


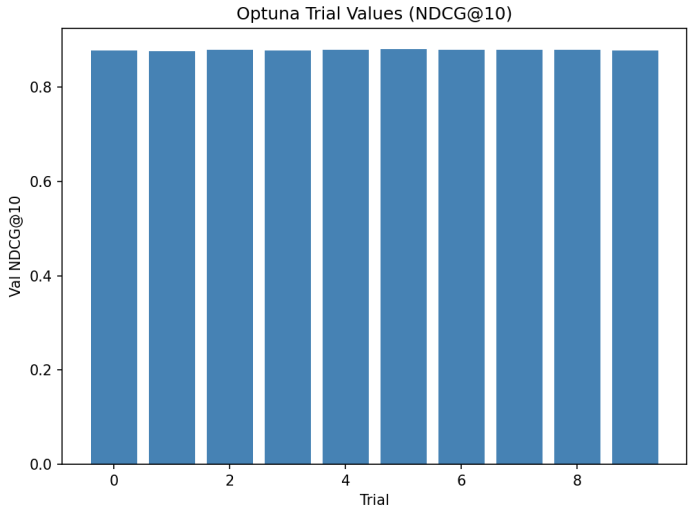


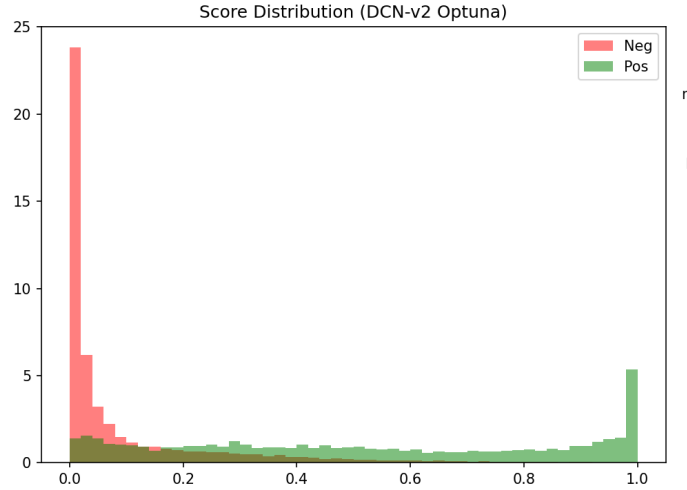


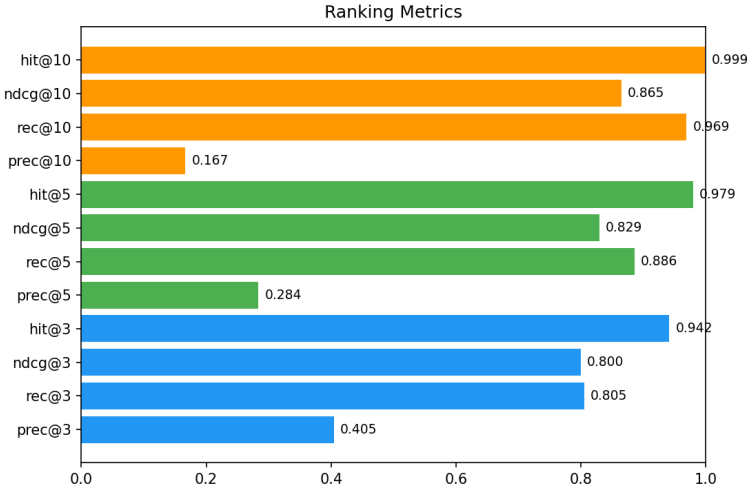


DCN-v2 Deep & Cross Network: Training Results

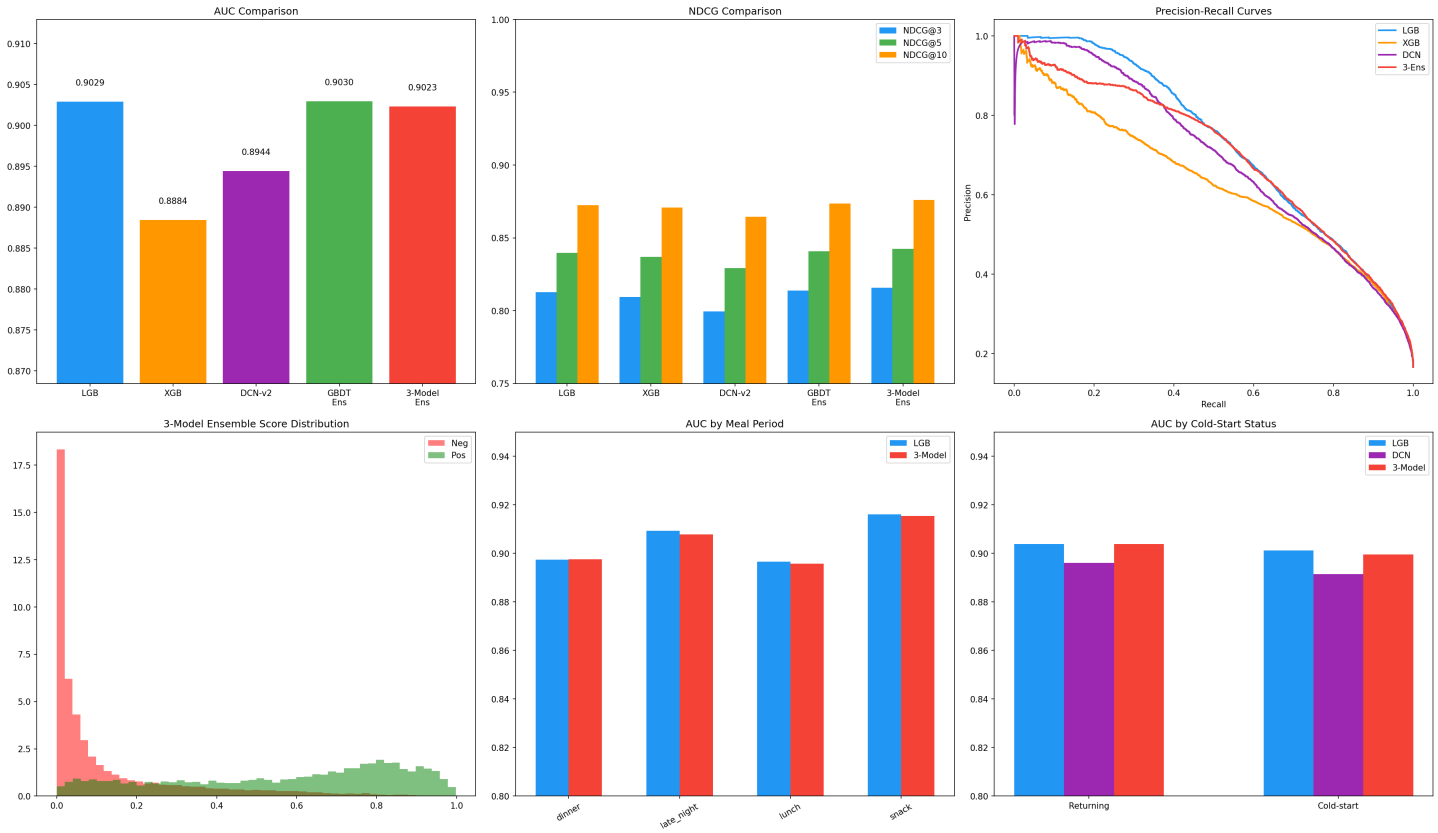








Final 3-Model Ensemble Comparison



References & Links

GitHub Repository:

https://github.com/Kashishgarg-15/Zomathon

Jupyter Notebook:

CSAO_Recommendation_System.ipynb (in repository root)

Key Files:

- train_model_v3.py - GBDT training with LLM features + Optuna tuning
- train_dcn_v2.py - DCN-v2 deep learning model
- final_ensemble_v2.py - 3-model ensemble
- generate_llm_features.py - Sentence-transformer feature engineering
- baseline_and_analysis.py - Baselines + business impact analysis
- inference_benchmark.py - Production latency benchmark
- DATA_DICTIONARY.md - Complete data pipeline documentation